

MODULE 09 · DOMAIN 3 - CLAUDE CODE CONFIGURATION & WORKFLOWS

# Extending Claude Code

Skills, hooks and plugins. Who invokes each one, and which of them can say no.

DOMAIN 3 of 5   EXAM WEIGHT 20%   MODULE 9 of 20

# Four extension points

| MECHANISM     | WHO INVOKES IT                          | CAN IT BLOCK?                     |
|---------------|-----------------------------------------|-----------------------------------|
| Skill         | You, or Claude, or both. You choose.    | No. It is instructions.           |
| Slash command | You. It is a skill with fewer features. | No.                               |
| Hook          | Nobody. It fires on an event.           | Yes, on the blocking events.      |
| Plugin        | It is a package of the other three.     | Only through the hooks inside it. |

## ON THE EXAM

Only one of these can say no. That single column decides most questions in this module.

Slash commands and skills have **merged**.  
One system, two directories, the same  
slash command.

# What a skill is

- ◆ A folder with a **SKILL.md** inside it, plus any supporting files, templates or scripts.
- ◆ **Markdown instructions** that Claude follows when the skill is active.
- ◆ **YAML frontmatter** controlling who can invoke it, which tools it may use, and where it runs.
- ◆ **Only the name and description sit in context until it is used.** The body loads on demand.

## ON THE EXAM

That last point is why skills scale where CLAUDE.md does not. Twenty skills cost twenty descriptions, not twenty documents.

# The frontmatter fields that earn marks

| FIELD                                 | WHAT IT DOES                                               |
|---------------------------------------|------------------------------------------------------------|
| <code>name</code>                     | Required. Becomes the slash command.                       |
| <code>description</code>              | Required. The text Claude matches against your intent.     |
| <code>disable-model-invocation</code> | Only you can invoke it. For anything with side effects.    |
| <code>user-invocable: false</code>    | Only Claude can invoke it. For background conventions.     |
| <code>allowed-tools</code>            | Restricts the skill to those tools, and pre-approves them. |
| <code>context: fork</code>            | Runs the skill in an isolated subagent.                    |

## ON THE EXAM

The description is not documentation. It is how Claude decides whether this skill is relevant.

# Who is allowed to invoke it?

DISABLE-MODEL-INVOCATION: TRUE

## Only you

- Claude will never trigger it on its own
- For anything with real side effects
- Deploys, releases, sending things
- You keep the trigger

USER-INVOCABLE: FALSE

## Only Claude

- No slash command is exposed
- For conventions Claude should just follow
- Background knowledge, applied when relevant
- Claude keeps the trigger

### THE TIEBREAKER

Default is both. Turn one off when the wrong invoker would be a problem.

# allowed-tools does two jobs

- ◆ **It restricts.** While the skill is active, Claude may use only those tools.
- ◆ **It pre-approves.** Listed tools do not prompt for permission each time.
- ◆ **It also shrinks context**, because only those tool definitions are included.
- ◆ **A research skill with read tools only cannot write**, whatever the instructions say.

## ON THE EXAM

This is capability scoping, not a hint. Same principle as restricting a subagent's tools in module 3.

# context: fork

- ◆ **Runs the skill inside an isolated subagent** with its own context window.
- ◆ **Verbose output stays in the fork.** The main conversation gets the result, not the working.
- ◆ **A forked skill has no conversation history.** Write it to be entirely self-contained.
- ◆ **For codebase analysis, brainstorming, research** — anything generating output you do not want to keep.

## ON THE EXAM

Everything from module 3 applies. Isolation is the benefit; nothing is inherited, and only the result returns.

A hook is not something Claude decides to run. It is something that **happens to** Claude.

# The five events that carry the work

| EVENT            | WHEN IT FIRES                        | CAN IT BLOCK?                             |
|------------------|--------------------------------------|-------------------------------------------|
| PreToolUse       | Before any tool call                 | Yes. This is the enforcement point.       |
| PostToolUse      | After a tool succeeds                | No. The tool already ran.                 |
| UserPromptSubmit | When you submit a prompt             | Yes. It can reject the prompt.            |
| Stop             | When Claude finishes a response turn | Yes. It can force Claude to keep working. |
| SessionStart     | At session start, and on resume      | No. Informational.                        |

## ON THE EXAM

The reference documents around thirty events. These five carry almost all real production use.

# Exit codes, and the footgun

- ◆ **Exit 0 allows.** The call proceeds normally.
- ◆ **Exit 2 blocks,** on the events that can block, and your stderr message goes back to Claude.
- ◆ **Exit 1 blocks nothing.** The conventional Unix failure code is the single biggest hook mistake.
- ◆ **A guardrail you never tested is worse than none,** because you have stopped watching for the thing it was meant to catch.

## ON THE EXAM

Pipe fake JSON into the script, check the exit code, then confirm from the transcript that a block actually blocks.

# PreToolUse or PostToolUse?

## PRETOOLUSE

### Prevention

- Sees the tool name and the exact arguments
- Exit 2 stops the call before it happens
- Where every guardrail belongs
- Cannot rewrite the arguments, only refuse

## POSTTOOLUSE

### Reaction

- Fires after the tool has already run
- Cannot undo anything
- For formatting, logging, notifying
- Can feed a message back to Claude

#### THE TIEBREAKER

"Prevent" means PreToolUse. "Whenever a file is written, also run the formatter" means PostToolUse.

# Four limitations worth knowing

- ◆ **Hooks cannot call tools or slash commands.** Exit codes, stdout and stderr, and nothing else.
- ◆ **PreToolUse cannot modify arguments.** Block and let Claude try again differently.
- ◆ **Stop fires on every response end,** not only when a task is complete. Gate logic must tolerate mid-task stops.
- ◆ **Hooks only cover what you thought of in advance.** They are a policy layer, not a safety net.

## ON THE EXAM

The last point is the honest one. A regex in a PreToolUse hook should not be the only thing between Claude and production.

Hooks **tighten** permissions. They never loosen them.

# Hook or permission rule?

## PERMISSION

### On or off

- Allow, deny or ask, matched on the tool
- No logic, no context, no conditions
- Cheap, declarative, easy to audit
- The right default for a flat rule

## HOOKE

### It can decide

- Sees the arguments, the path, the time
- Runs code, so it can evaluate anything
- For conditional rules a permission cannot express
- You own the correctness of the logic

## THE TIEBREAKER

Never read .env is a permission. Never write to production outside the change window is a hook.

# Plugins: packaging, not a new capability

- ◆ A plugin bundles skills, hooks, subagents and MCP servers into one installable unit.
- ◆ For workflows shared across repositories. A /release skill used in five codebases belongs in a plugin.
- ◆ Installed from a marketplace or a URL, with discovery and updates handled for you.
- ◆ Project skills stay in the repository when the workflow is specific to that codebase.

## ON THE EXAM

Plugin for cross-repo reuse. Project skill for this codebase. The axis is distribution, not capability.

# Choosing the mechanism

| WHAT YOU NEED                                 | WHAT TO USE                             |
|-----------------------------------------------|-----------------------------------------|
| Claude should know this, always               | CLAUDE.md, or a path-scoped rule        |
| A repeated request, run on demand             | A skill, invoked as a slash command     |
| A workflow with steps, assets and tool limits | A skill with allowed-tools              |
| A noisy task that should not pollute context  | A skill with context: fork              |
| Something must happen on every tool call      | A PostToolUse hook                      |
| Something must never happen                   | A permission deny, or a PreToolUse hook |
| The same setup across many repositories       | A plugin                                |

## ON THE EXAM

Read the row you need, then check the verb. "Should" is a skill. "Must" is a hook or a permission.

# Currency check: what has moved

| TOPIC                 | EXAM ANSWER                   | CURRENT REALITY                                                      |
|-----------------------|-------------------------------|----------------------------------------------------------------------|
| Commands and skills   | Two separate mechanisms       | Merged. Both create the same slash command; skills win a name clash. |
| Hook events           | A handful of lifecycle events | Around thirty documented, and still growing.                         |
| Skills in frontmatter | Not covered                   | Skills can carry their own hooks, scoped to the skill's lifetime.    |

## ON THE EXAM

Answer the guide's framing on the exam. None of these changes make a guide-era answer wrong, only incomplete.

# Module 9 in five lines

- ◆ Skills are instructions with frontmatter. Only the name and description cost context until they are used.
- ◆ allowed-tools restricts and pre-approves. context: fork isolates and returns only the result.
- ◆ Hooks fire on events, not on Claude's decision. Exit 2 blocks; exit 1 blocks nothing.
- ◆ PreToolUse prevents. PostToolUse reacts and cannot undo.
- ◆ Permissions are flat rules. Hooks are conditional ones. Plugins are distribution.

## ON THE EXAM

Five scenarios next. Two of them look identical and want opposite hooks.

A team requires that every Python file Claude writes or edits is formatted with their standard formatter before it is committed.

The rule is currently in CLAUDE.md. Compliance is roughly 90%. When Claude is deep in a multi-file change it sometimes skips the formatting step.

They want it to happen every time, and they do not want Claude to have to remember.

## What should they implement?

- ☐ A A PreToolUse hook on Write and Edit that blocks unformatted content
- ☐ B A PostToolUse hook on Write and Edit that runs the formatter
- ☐ C A skill called /format that Claude invokes after editing files
- ☐ D A permissions rule denying Write unless the content is formatted

### ◆ PAUSE AND DECIDE

## What should they implement?

- A

 PreToolUse blocking  
*Blocks the write and makes Claude reformulate. It turns a formatting step into a retry loop for no benefit.*
- B

 PostToolUse formatter  
*The file is written, then formatted, every time. Claude never has to remember anything.*
- C

 A /format skill  
*Still relies on Claude choosing to invoke it. That is the 10% failure they already have.*
- D

 Conditional permission  
*Permissions match on the tool, not on content. They cannot express "unless formatted".*

### THE SIGNAL

The requirement is "also do this", not "never do that". Additive rules are PostToolUse; prohibitions are PreToolUse.

The same team also requires that Claude never runs a database migration command against any environment whose name contains "prod", under any circumstances.

Migration commands are run through Bash and the environment appears as a flag in the command string.

Some engineers run Claude Code with permission prompts bypassed.

### What should they implement?

- ☐ A A PostToolUse hook on Bash that alerts when a production migration is detected
- ☐ B A PreToolUse hook on Bash that inspects the command and exits 2 on a match
- ☐ C A permissions deny rule on Bash
- ☐ D A CLAUDE.md rule, since the team already follows their conventions closely

### ◆ PAUSE AND DECIDE

## What should they implement?

**A** PostToolUse alert

*The migration already ran. An alert tells you about the outage you are now having.*

**C** Deny Bash

*Permissions match the tool, not the command's contents. Denying Bash outright removes every legitimate use.*

**B** PreToolUse, exit 2

*Inspects the command string, blocks before execution, and holds even with permission prompts bypassed.*

**D** A CLAUDE.md rule

*Context. "Under any circumstances" rules it out before you finish reading it.*

### THE SIGNAL

Pair this with scenario 1. Same team, same tools, opposite hooks, because one requirement adds and the other forbids.

An engineer has written a skill that audits a codebase for unused dependencies. It reads package files, greps across the repository, and produces a short report.

It works, but running it fills a large portion of the context window with file listings and grep output. Afterwards Claude is noticeably worse at the task the engineer was actually working on.

The report itself is about fifteen lines.

### What is the most effective change?

- ☐ A Add context: fork to the skill's frontmatter
- ☐ B Add allowed-tools listing only Read and Grep
- ☐ C Add disable-model-invocation so it runs less often
- ☐ D Move the skill into a plugin so it loads separately

### ◆ PAUSE AND DECIDE

## What is the most effective change?

- A

 context: fork  
*The exploration happens in an isolated window and only the fifteen-line report returns.*
- B

 allowed-tools  
*Restricts capability and trims tool definitions. The grep output is still enormous and still returns.*
- C

 disable-model-invocation  
*Runs the same problem less frequently. The engineer wants to run it.*
- D

 Into a plugin  
*Plugins are distribution. Packaging a skill differently does not change what it puts in context.*

### THE SIGNAL

"Fifteen lines of output, a context window of working" is the fork signature exactly.

A platform team maintains a release workflow used identically across eleven service repositories. It consists of three skills, a PreToolUse hook, and an MCP server configuration.

Today it is copy-pasted into each repository's .claude directory. Updates require eleven pull requests, and three repositories are already out of date.

### What should they do?

- ☐ A Move it to user-level configuration so each engineer has it everywhere
- ☐ B Package it as a plugin and install it in each repository
- ☐ C Put it in a shared repository and import it with @path from each CLAUDE.md
- ☐ D Move it into the enterprise managed settings file

### ◆ PAUSE AND DECIDE

## What should they do?

- A

 User-level config  
*Per engineer, not per repository. Every new joiner sets it up by hand, and it applies to unrelated projects too.*
- B

 Package as a plugin  
*A plugin is exactly a bundle of skills, hooks and MCP config, with discovery and updates handled.*
- C

 Shared repo plus imports  
*Imports pull in memory file content. They do not distribute hooks or MCP server configuration.*
- D

 Enterprise managed settings  
*That layer is for policy that must not be overridden. This is a workflow, not a restriction.*

### THE SIGNAL

The contents list is the answer. Skills plus a hook plus MCP configuration is the definition of a plugin.

A team has a skill that deploys to staging. It has side effects and takes about four minutes. Engineers report that Claude sometimes invokes it on its own, after interpreting a question like "would this work on staging?" as a request to deploy.

They still want to be able to run it themselves with a slash command.

### What is the correct configuration?

- ☐ A Set user-invocable: false
- ☐ B Set disable-model-invocation: true
- ☐ C Add a PreToolUse hook that blocks the deploy command
- ☐ D Rewrite the description so Claude matches it less often

### ◆ PAUSE AND DECIDE

## What is the correct configuration?

- A

`user-invocable: false`  
*Exactly backwards. That removes the slash command and leaves Claude as the only invoker.*
- B

`disable-model-invocation: true`  
*Claude can no longer trigger it; the slash command stays. Both requirements, one field.*
- C

`PreToolUse` block  
*Would block the engineers' own deploys too. The hook cannot tell who asked.*
- D

 Reword the description  
*Reduces the match rate. The requirement is that Claude never triggers it, not that it rarely does.*

### THE SIGNAL

Two fields, opposite meanings, similar names. Read which invoker the scenario wants removed.